

Slot 3 · Write the label function

File: `label_fn.py` · **Done when:** `uv run pytest tests/test_slot03.py` shows 5 passed · **Time:** 18 min

Today's idea

The model learns from a **label**: here, "how many minutes did this delivery really take?" It sounds easy: `delivered_at - placed_at`. It is not. Real data has holes and traps. Every row you throw away is a decision. Today you make those decisions.

Step 1 · The task

Open `label_fn.py`. Write `make_label(orders)`:

- **In:** the raw orders table (412,000 rows)
- **Out:** only the rows you trust, with a new column `actual_minutes`

Start simple. Turn the text timestamps into times and subtract:

```
def make_label(orders):
    ts = lambda c: pl.col(c).str.to_datetime(time_unit="us")
    return orders.with_columns(
        ((ts("delivered_at") - ts("placed_at")).dt.total_seconds() / 60)
        .alias("actual_minutes")
    )
```

Step 2 · Run it, look, fix, repeat

```
uv run python label_fn.py
```

It prints a summary of `actual_minutes` and how many rows you kept. **Look at min, max and null_count.** Does each one make sense for a food delivery?

Things to ask yourself, one at a time:

- Some rows have no `delivered_at`. Why? (Look at the `status` column.)
- Some minutes are **negative**. How can a delivery take minus five hours?
- Can a delivery be `delivered_at` **before** `picked_up_at`?
- Can one order appear twice with different `order_id`s?

Useful polars pieces:

```
orders["status"].value_counts()
d.filter(pl.col("status") == "delivered")
d.filter(pl.col("actual_minutes") < 0).head()
pl.col("placed_at_parsed").dt.offset_by("-90m")    # shift a time
```

Stuck? Stuck on the negative minutes for more than 5 minutes? Run `uv run python scripts/hint_slot03.py`. Look at which row is different.

Careful Throwing away a whole city is **not** a fix. The test checks for this.

Step 3 · Swap with a neighbour

Run your neighbour's `make_label` on your data (copy their function into a scratch file). Compare how many rows each of you kept. Same data — different training sets!

Step 4 · Count your drops out loud

Write down how many rows each decision removed. The reference answer keeps **366,061 of 412,000 (11.2% dropped)**. Anything from 80% to 95% kept passes the test.

Think Which of your decisions **dropped nothing** but saved a lot of rows?

Step 5 · Check you are done

```
uv run pytest tests/test_slot03.py
```

If something goes wrong

what you see	what to do
kept more than 95%	You still have negative or empty labels. Check <code>actual_minutes.min()</code> and <code>null_count()</code> .
kept less than 80%	You dropped too much. Is Pune still about 16% of your rows?
<code>str.to_datetime</code> error	Empty values in the column. Filter the status first, then parse.
moved the time the wrong way	The wrong clock is ahead of UTC. Subtract, don't add.

Note If you cannot finish, don't worry. Slots 4 onwards use `eta/label.py`, not your file, so nobody is blocked.